

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

**ANALYSIS AND DESIGN OF ALGORITHMS
(23CS4PCADA)**

Submitted by

**AMRUTA SHARANABASAPPA NIMBAL
(1BM24CS037)**

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **Amruta Sharanabasappa Nimbal (1BM24CS037)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Kanchana M Dixit
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	6 – 8
2	Implement Johnson Trotter algorithm to generate permutations.	11 – 13
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	14 – 17
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	18 – 20
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	21 – 23
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	24 – 26
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	28 – 29
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	31 – 36
9	Implement Fractional Knapsack using Greedy technique.	37 – 39
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	40 – 42
11	Implement "N-Queens Problem" using Backtracking.	43 – 45

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Leetcode 11:

The screenshot shows the LeetCode problem page for "11. Container With Most Water". The problem description states: "You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the i^{th} line are $(i, 0)$ and $(i, height[i])$. Find two lines that together with the x-axis form a container, such that the container contains the most water. Return the maximum amount of water a container can store. Notice that you may not slant the container."

Example 1:

The diagram shows a bar chart with heights [1, 8, 6, 2, 5, 4, 3, 7]. The area between the line at index 1 (height 8) and index 7 (height 7) is shaded blue, representing the maximum water capacity.

The code editor shows the following C++ solution:

```
int maxArea(vector<int> height, int heightSize) {
    int left = 0;
    int right = heightSize - 1;
    int maxArea = 0;

    while (left < right) {
        int h = min(height[left], height[right]);
        int area = h * (right - left);
        if (area > maxArea)
            maxArea = area;

        if (height[left] < height[right])
            left++;
        else
            right--;
    }

    return maxArea;
}
```

The test result shows "Accepted" with a runtime of 0 ms.

The screenshot shows the LeetCode submission page for "11. Container With Most Water". The submission is marked as "Accepted" with a runtime of 0 ms. The input is [1, 8, 6, 2, 5, 4, 3, 7] and the output is 49.

The runtime and memory usage are shown as:

- Runtime: 0 ms, Beats 100.00%
- Memory: 14.05 MB, Beats 9.34%

The code editor shows the following C++ solution:

```
int maxArea(vector<int> height, int heightSize) {
    int left = 0;
    int right = heightSize - 1;
    int maxArea = 0;

    while (left < right) {
        int h = min(height[left], height[right]);
        int area = h * (right - left);
        if (area > maxArea)
            maxArea = area;

        if (height[left] < height[right])
            left++;
        else
            right--;
    }

    return maxArea;
}
```

The test result shows "Accepted" with a runtime of 0 ms.

Leetcode 34:

The screenshot shows the LeetCode problem page for "34. Find First and Last Position of Element in Sorted Array". The problem description states: "Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given `target` value. If `target` is not found in the array, return `[-1, -1]`. You must write an algorithm with $O(\log n)$ runtime complexity."

Example 1:
Input: `nums = [5,7,7,8,8,10]`, `target = 8`
Output: `[3,4]`

Example 2:
Input: `nums = [5,7,7,8,8,10]`, `target = 6`
Output: `[-1,-1]`

Example 3:
Input: `nums = []`, `target = 0`
Output: `[-1,-1]`

The code editor shows a C++ solution using binary search:

```
int* searchRange(int* nums, int numsSize, int target, int* returnSize) {
    int first = -1;
    int last = -1;
    for(int i=0; i<numsSize; i++)
        if(nums[i] == target){
            if(first == -1)
                first = i;
            last = i;
        }
    *returnSize = 2;
    return result;
}
```

The test results show "Accepted" with a runtime of 0 ms.

The screenshot shows the submission page for the same problem. It displays the submission status as "Accepted" with a runtime of 0 ms and memory usage of 10.50 MB. A bar chart shows the submission performance relative to other users.

Input:
`nums = [5,7,7,8,8,10]`
`target = 8`

Output:
`[3,4]`

Expected:
`[3,4]`

The code editor shows the same C++ solution as in the previous screenshot.

Lab Program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>

int graph[10][10], visited[10], stack[10];

int n, top = -1;

void dfs(int v)
{
    visited[v] = 1;
    for(int i = 0; i < n; i++)
    {
        if(graph[v][i] == 1 && !visited[i])
        {
            dfs(i);
        }
    }
    stack[++top] = v;
}

int main()
{
    int edges, u, v;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            graph[i][j] = 0;
        }
    }
}
```

```

    }
}
printf("Enter number of edges: ");
scanf("%d", &edges);
printf("Enter edges (u v):\n");
for(int i = 0; i < edges; i++)
{
    scanf("%d %d", &u, &v);
    graph[u][v] = 1;
}
for(int i = 0; i < n; i++)
{
    if(!visited[i])
    {
        dfs(i);
    }
}
printf("Topological Sorting:\n");
while(top != -1)
{
    printf("%d ", stack[top--]);
}
return 0;
}

```

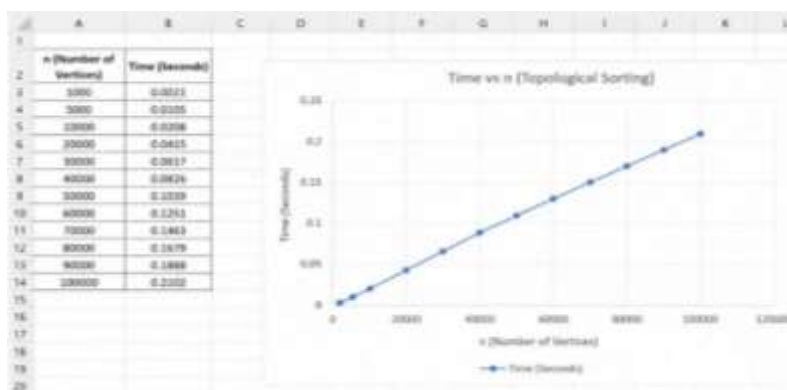
Output:

```

C:\Program Files\Java\jdk-11.0.2\bin\java.exe
Enter number of vertices: 4
Enter number of edges: 4
Enter edges (u v):
0 2
0 3
0 0
0 1
1 3
1 1
Topological Sorting:
0 1 2 3 0
Process returned 0 (0x0)   execution time : 31,160 s
Press any key to continue.

```

Time Complexity: $O(V+E)$



Leetcode 268:

The screenshot shows the LeetCode problem page for "268. Missing Number". The problem description states: "Given an array *nums* containing *n* distinct numbers in the range $[0, n]$, return the only number in the range that is missing from the array."

Example 1:
Input: *nums* = [3, 0, 1]
Output: 2
Explanation: $n = 3$, since there are 3 numbers, so all numbers are in the range $[0, 3]$. 2 is the missing number in the range since it does not appear in *nums*.

Example 2:
Input: *nums* = [0, 1]
Output: 2
Explanation: $n = 2$, since there are 2 numbers, so all numbers are in the range $[0, 2]$. 2 is the missing number in the range since it does not appear in *nums*.

The C++ code provided is:

```
int missingNumber(int* nums, int numsSize) {
    int n = numsSize * (numsSize + 1) / 2;
    int sum = 0;
    for(int i=0; i<numsSize; i++)
        sum += nums[i];
    return n - sum;
}
```

The solution is marked as "Accepted" with a runtime of 0 ms and memory usage of 10.63%.

This screenshot shows the same problem page but with more details about the accepted solution. The "Accepted" status is confirmed with a runtime of 0 ms and memory usage of 10.63%. The input and output are shown as:

Input: *nums* = [3, 0, 1]
Output: 2
Expected: 2

The C++ code is repeated at the bottom of the page.

Leetcode 1636:

The screenshot shows the LeetCode problem page for "1636. Sort Array by Increasing Frequency". The problem description states: "Given an array of integers `nums`, sort the array in increasing order based on the frequency of the values. If multiple values have the same frequency, sort them in decreasing order." The example provided is: Input: `nums = [1,1,2,2,2,3]`, Output: `[3,1,1,2,2,2]`. The explanation notes that '3' has a frequency of 1, '1' has a frequency of 2, and '2' has a frequency of 3. The code editor shows a C++ solution using a map to count frequencies and a custom comparator to sort the array based on these frequencies.

1636. Sort Array by Increasing Frequency

Given an array of integers `nums`, sort the array in increasing order based on the frequency of the values. If multiple values have the same frequency, sort them in decreasing order.

Return the sorted array.

Example 1:

Input: `nums = [1,1,2,2,2,3]`
Output: `[3,1,1,2,2,2]`
Explanation: '3' has a frequency of 1, '1' has a frequency of 2, and '2' has a frequency of 3.

Example 2:

Input: `nums = [2,3,3,3,2,1]`
Output: `[1,2,3,3,3,2]`
Explanation: '2' and '3' both have a frequency of 2, so they are sorted in decreasing order.

Example 3:

Input: `nums = [-1,1,-6,4,5,-6,1,4,1]`
Output: `[5,-1,-1,4,4,-6,-6,1,1,1]`

```
int freq[201];
int compare(const void* a, const void* b) {
    int x = *(int*)a;
    int y = *(int*)b;
    if (freq[x + 100] == freq[y + 100]) {
        return y < x; // Same Frequency -> Larger Number First
    }
    return freq[x + 100] < freq[y + 100]; // Same Frequency First
}

int* frequencySort(int* nums, int numsSize, int* returnSize) {
    *returnSize = numsSize;
    for (int i = 0; i < 201; i++)
        freq[i] = 0;
    for (int i = 0; i < numsSize; i++)
        freq[nums[i] + 100]++;
    qsort(nums, numsSize, sizeof(int), compare);
    return nums;
}
```

Accepted Runtime: 0 ms

The screenshot shows the LeetCode submission page for the same problem. It displays the accepted status, runtime (0 ms), and memory usage (12.02 MB / 31.86%). A bar chart shows the performance of the submission compared to other submissions. The code editor shows the same C++ solution. The input and output fields are filled with the example input and output.

Accepted Runtime: 0 ms

Accepted 100.00%

12.02 MB / 31.86%

Accepted

Case 1 Case 2 Case 3

Input: `nums = [1,1,2,2,2,3]`

Output: `[3,1,1,2,2,2]`

Expected: `[3,1,1,2,2,2]`

Contribute to LeetCode

Lab Program 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>
```

```
void swap(int *a, int *b)
```

```
{  
    int temp;  
    temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
void permute(int a[], int start, int end)
```

```
{  
    int i;  
    if(start == end)  
    {  
        for(i = 0; i <= end; i++)  
        {  
            printf("%d ", a[i]);  
        }  
        printf("\n");  
    }  
    else  
    {  
        for(i = start; i <= end; i++)  
        {  
            swap(&a[start], &a[i]);  
            permute(a, start + 1, end);  
        }  
    }  
}
```

```
        swap(&a[start], &a[i]);
    }
}
}

int main()
{
    int n, i;
    int a[10];
    printf("Enter n: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        a[i] = i + 1;
    }
    printf("All permutations are:\n");
    permute(a, 0, n - 1);
    return 0;
}
```

Output:

```
Enter n: 6
All permutations are:
1 2 3 4
1 2 4 3
1 3 2 4
1 3 4 2
1 4 2 3
1 4 3 2
2 1 3 4
2 1 4 3
2 3 1 4
2 3 4 1
2 4 1 3
2 4 3 1
3 1 2 4
3 1 4 2
3 2 1 4
3 2 4 1
3 4 1 2
3 4 2 1
4 1 2 3
4 1 3 2
4 2 1 3
4 2 3 1
4 3 1 2
4 3 2 1
Process returned 0 (0x0)   execution time : 0.308 s
Press any key to continue.
```

Lab Program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<time.h>
```

```
void merge(int a[], int f, int m, int l)
```

```
{
```

```
    int i, j, k;
```

```
    i = f;
```

```
    j = m + 1;
```

```
    k = 0;
```

```
    int r[100];
```

```
    while(i <= m && j <= l)
```

```
    {
```

```
        if(a[i] < a[j])
```

```
        {
```

```
            r[k] = a[i];
```

```
            i++;
```

```
        }
```

```
        else
```

```
        {
```

```
            r[k] = a[j];
```

```
            j++;
```

```
        }
```

```
        k++;
```

```
    }
```

```
    while(i <= m)
```

```
    {
```

```

        r[k] = a[i];
        i++;
        k++;
    }
    while(j <= l)
    {
        r[k] = a[j];
        j++;
        k++;
    }
    k = 0;
    for(i = f; i <= l; i++)
    {
        a[i] = r[k];
        k++;
    }
}

void mergesort(int a[], int f, int l)
{
    if(f < l)
    {
        int m = (f + l) / 2;
        mergesort(a, f, m);
        mergesort(a, m + 1, l);
        merge(a, f, m, l);
    }
}

```

```

int main()
{
    int n, i;
    clock_t start, end;
    double cpu_time;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    start = clock();
    mergesort(a, 0, n - 1);
    end = clock();
    cpu_time =
    ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nSorted elements:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");
    printf("Time taken = %f seconds\n", cpu_time);
    return 0;
}

```

Output:


```

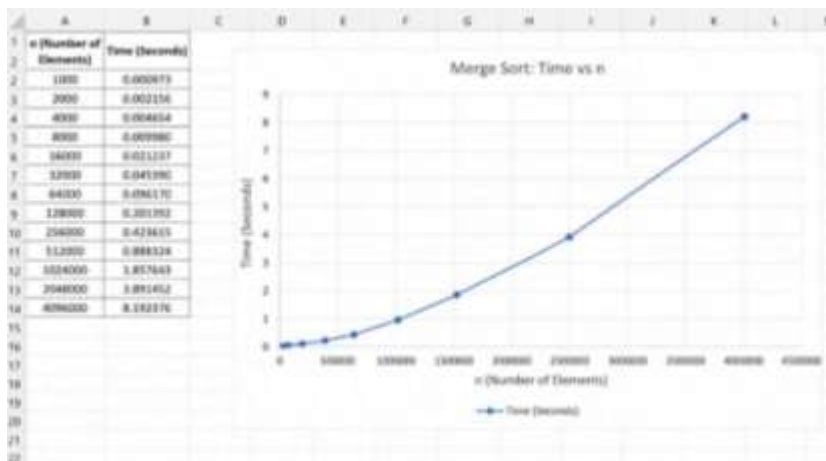
C:\Users\user\Documents> g++ mergeSort.cpp -std=c++11
Enter number of elements: 5
Enter elements:
12345
6789
1011
1213
1415

Sorted elements:
12345 1011 6789 1213 1415
Time taken = 0.000000 seconds

Process returned 0 (0x0)   execution time : 0.002 s
Press any key to continue.

```

Time Complexity: $O(n \log n)$



Lab Program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<time.h>
```

```
void swap(int *a, int *b)
```

```
{
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int partition(int a[], int low, int high)
```

```
{
```

```
    int pivot = a[low];
```

```
    int i = low+1;
```

```
    int j = high;
```

```
    while(1)
```

```
    {
```

```
        while(i<=high && a[i]<=pivot)
```

```
            i++;
```

```
        while(a[j]>pivot)
```

```
            j--;
```

```
        if(i<j)
```

```
            swap(&a[i], &a[j]);
```

```
        else
```

```
            break;
```

```
    }
```

```

        swap(&a[low], &a[j]);
        return j;
    }

void quicksort(int a[], int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);
        quicksort(a, low, p - 1);
        quicksort(a, p + 1, high);
    }
}

int main()
{
    int n, i;
    clock_t start, end;
    double cpu_time;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
    start = clock();
    quicksort(a, 0, n - 1);
    end = clock();
    cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Sorted elements:\n");
    for(i = 0; i < n; i++)

```

```

        printf("%d ", a[i]);

    printf("\nTime taken = %f seconds\n", cpu_time);

    return 0;
}

```

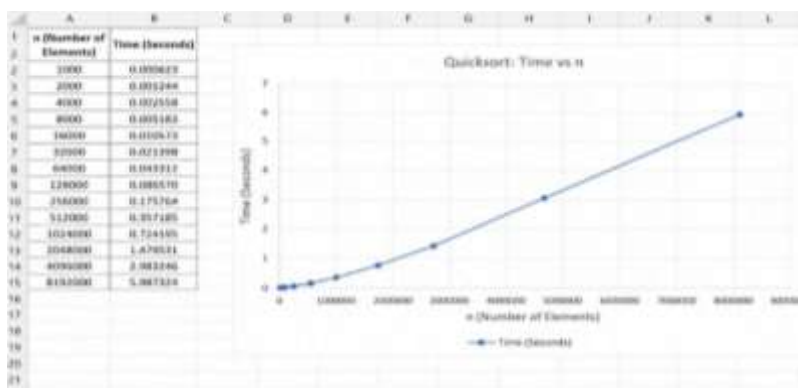
Output:

```

C:\Users\user> g++ 1.1.cpp
Enter the number of elements: 5
Enter elements:
27
58
29
13
6
Sorted elements:
5 11 27 58 59
Time taken = 0.000000 seconds
Process returned 0 (0x0)   execution time : 0.075 s
Press any key to continue.

```

Time Complexity: $O(n \log n)$



Lab Program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include<stdio.h>
```

```
void heapify(int a[], int n, int p)
```

```
{  
    int c, item;  
    item = a[p];  
    c = 2*p+1;  
    while(c<=(n-1))  
    {  
        if((c+1<n) && (a[c]<a[c+1]))  
            c++;  
        if(item<a[c])  
        {  
            a[p] = a[c];  
            p = c;  
            c = 2*p+1;  
        }  
        else  
            break;  
    }  
    a[p] = item;  
}
```

```
void build_heap(int a[], int n)
```

```
{  
    int i;  
    for(i=(n-1)/2; i>=0; i--)
```

```

        heapify(a, n, i);
    }

void heap_sort(int a[], int n)
{
    int i, temp;
    build_heap(a, n);
    for(i=(n-1); i>0; i--)
    {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;
        heapify(a, i, 0);
    }
}

int main()
{
    int n, a[20], i;
    printf("Enter the number of elements:");
    scanf("%d", &n);
    printf("Enter elements:");
    for(i=0; i<n; i++)
        scanf("%d", &a[i]);
    heap_sort(a, n);
    printf("Heap Sorted elements:");
    for(i=0; i<n; i++)
        printf("%d\t", a[i]);
    return 0;
}

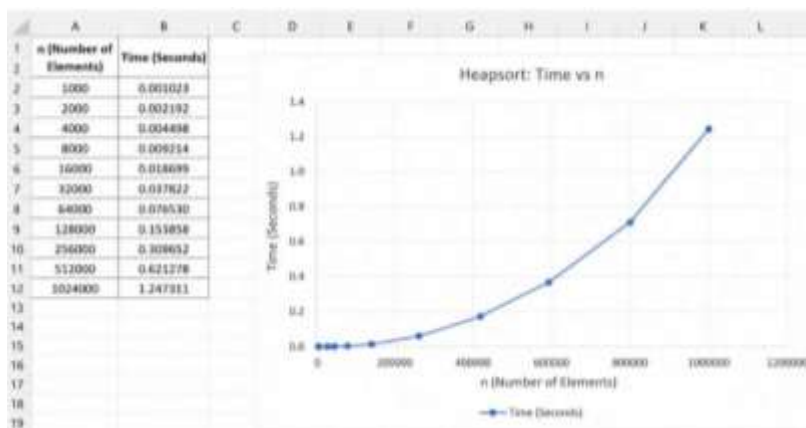
```

}

Output:

```
Enter the number of elements: 9
Enter elements: 15 11 8 50 19 20 3 12
Heap Sorted elements: 3 9 11 12 15 19 20 50 79
Process returned 0 (0x0)    execution time : 12.388 s
Press any key to continue.
```

Time Complexity: $O(n \log n)$



Lab Program 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>
```

```
struct Item
```

```
{  
    int weight;  
    int profit;  
};
```

```
int max(int a, int b)
```

```
{  
    if(a > b)  
        return a;  
    else  
        return b;  
}
```

```
int main()
```

```
{  
    int n, capacity;  
    printf("Enter number of items: ");  
    scanf("%d", &n);  
    struct Item item[n];  
    for(int i = 0; i < n; i++)  
    {  
        printf("Enter weight and profit of item %d: ", i + 1);  
        scanf("%d %d", &item[i].weight, &item[i].profit);  
    }
```



```

printf("Enter knapsack capacity: ");
scanf("%d", &capacity);
int dp[n + 1][capacity + 1];
for(int i = 0; i <= n; i++)
{
    for(int w = 0; w <= capacity; w++)
    {
        if(i == 0 || w == 0)
        {
            dp[i][w] = 0;
        }
        else if(item[i - 1].weight <= w)
        {
            dp[i][w] = max(
                item[i - 1].profit +
                dp[i - 1][w - item[i - 1].weight],
                dp[i - 1][w]
            );
        }
        else
        {
            dp[i][w] = dp[i - 1][w];
        }
    }
}
printf("Maximum Profit = %d\n", dp[n][capacity]);
return 0;
}

```

Output:

```
Enter number of items: 4
Enter weight and profit of item 1: 5 Y
Enter weight and profit of item 2: 3 N
Enter weight and profit of item 3: 4 N
Enter weight and profit of item 4: 1 Y
Enter knapsack capacity: Y
Maximum Profit = 8

Process returned 0 (0x0)   execution time : 34.565 s
Press any key to continue.
```

Leetcode 801:

The screenshot shows the LeetCode problem page for "801. Minimum Swaps To Make Sequences Increasing". The problem description states: "You are given two integer arrays of the same length `nums1` and `nums2`. In one operation, you are allowed to swap `nums1[i]` with `nums2[i]`." It provides an example: "For example, if `nums1 = [1,2,3,4]`, and `nums2 = [5,4,3,2]`, you can swap the element at `i = 1` to obtain `nums1 = [1,2,3,4]` and `nums2 = [5,4,3,2]`." The goal is to "Return the minimum number of needed operations to make `nums1` and `nums2` strictly increasing." It also defines a strictly increasing array: "An array `arr` is strictly increasing if and only if `arr[0] < arr[1] < arr[2] < ... < arr[arr.length - 1]`." Example 1: "Input: `nums1 = [1,2,3,4]`, `nums2 = [1,2,3,7]`. Output: `1`. Explanation: Swap `nums1[3]` and `nums2[3]`. Thus the sequences become: `nums1 = [1, 2, 3, 7]` and `nums2 = [1, 2, 3, 4]` which are both strictly increasing."

```
1 int minSwaps(vector<int> &nums1, vector<int> &nums2) {
2     return minSwaps(nums1, nums2, 0);
3 }
4
5 int minSwaps(vector<int> &nums1, vector<int> &nums2, int i, int swaps) {
6     if (i == nums1.size()) return swaps;
7     int swap = 0; // swap at current position
8     int noSwap = 0; // no swap at current position
9
10    for (int j = i; j < nums1.size(); j++) {
11        int noSwap = 1000000;
12        int swap = 1000000;
13
14        if (nums1[i] < nums1[j] && (i == 0 || nums1[i-1] < nums1[j])) {
15            noSwap = min(noSwap, noSwap);
16            swap = min(noSwap, swap + 1);
17        }
18        if (nums1[i] < nums1[j] && (i == 0 || nums1[i-1] < nums2[j])) {
19            noSwap = min(noSwap, noSwap);
20            swap = min(noSwap, swap + 1);
21        }
22    }
23    return min(noSwap, swap);
24 }
```

The screenshot shows the LeetCode submission page for "801. Minimum Swaps To Make Sequences Increasing". The submission is marked as "Accepted" with a runtime of 0 ms and memory of 16.35 MB. The test results show that the submission passed all 117/117 test cases. The input and output are displayed: "Input: `nums1 = [1,2,3,4]`, `nums2 = [1,2,3,7]`. Output: `1`. Expected: `1`." The submission was made by "Ananta_Mishra" on April 06, 2020, at 19:44.

Lab Program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include<stdio.h>

#include<math.h>

int main()
{
    int i, j, n, k;
    int cost[20][20];
    printf("Enter number of vertices:");
    scanf("%d", &n);
    printf("Enter cost matrix:");
    for(i=0; i<n; i++)
    {
        for(j=0; j<n; j++)
            scanf("%d", &cost[i][j]);
    }
    for(k=0; k<n; k++)
    {
        for(i=0; i<n; i++)
        {
            for(j=0; j<n; j++)
            {
                if(cost[i][j] > cost[i][k]+cost[k][j])
                    cost[i][j] = cost[i][k]+cost[k][j];
            }
        }
    }
    printf("\nShortest path matrix:\n");
```

```

for(i=0; i<n; i++)
{
    for(j=0; j<n; j++)
        printf("%d\t", cost[i][j]);

    printf("\n");
}

return 0;
}

```

Output:

```

C:\Users\user> g++ test.cpp
Enter number of vertices:4
Enter cost matrix:
0 8 9 7
8 0 7 4
9 7 0 1
7 9 9 0

Shortest path matrix:
0 8 9 7
8 0 7 4
9 7 0 1
7 9 9 0

Process returned 0 (Exit)   execution time : 23.111 s
Press any key to continue.

```

Leetcode 287:

The screenshot shows the LeetCode problem page for "287. Find the Duplicate Number". The problem description states: "Given an array of integers `nums` containing $n + 1$ integers where each integer is in the range $[1, n]$ inclusive. There is only one repeated number in `nums`, return the repeated number. You must solve the problem without modifying the array `nums` and using only constant extra space." Examples provided are: Example 1: Input: `nums = [1,3,4,2,2]`, Output: `2`; Example 2: Input: `nums = [1,1,3,4,2]`, Output: `1`; Example 3: Input: `nums = [2,3,3,3,3]`, Output: `3`. The code editor shows a C++ solution using a hash table:

```
int findDuplicate(int* nums, int numsSize) { int hashTable[numsSize]; for(int i=0; i<numsSize; i++){ hashTable[i]=0; } for(int i=0; i<numsSize; i++){ hashTable[nums[i]]++; } for(int i=0; i<numsSize; i++){ if(hashTable[i]>1){ return i; } } return -1; }
```

 The test result shows "Accepted" for all three cases.

The screenshot shows the submission page for the same problem. It displays the submission status as "Accepted" with a runtime of 3 ms (76.54%) and memory usage of 15.02 MB (15.76%). A bar chart shows the performance relative to other submissions. The code editor shows the same C++ solution. The test result section shows the input `[1,3,4,2,2]`, the output `2`, and the expected output `2`.

Lab Program 8:

(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>

#define INF 999

int main()
{
    int n;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    int cost[10][10];

    printf("Enter cost matrix:\n");

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);

            if(cost[i][j] == 0 && i != j)
            {
                cost[i][j] = INF;
            }
        }
    }

    int visited[10] = {0};
    visited[0] = 1;

    int edges = 0;

    int minCost = 0;

    printf("\nEdges in MST:\n");
```

```

while(edges < n - 1)
{
    int min = INF;
    int u = -1, v = -1;
    for(int i = 0; i < n; i++)
    {
        if(visited[i])
        {
            for(int j = 0; j < n; j++)
            {
                if(!visited[j] && cost[i][j] < min)
                {
                    min = cost[i][j];
                    u = i;
                    v = j;
                }
            }
        }
    }
    printf("%d -> %d = %d\n", u, v, min);
    minCost += min;
    visited[v] = 1;
    edges++;
}
printf("\nMinimum Cost = %d\n", minCost);
return 0;
}

```

Output:


```
Enter number of vertices: 5
Enter cost matrix:
0 20 5 0 0 10 0
20 0 10 0 0 0 10
0 10 0 10 0 0 0
0 0 10 0 10 0 10
0 0 0 10 0 10 0
10 0 0 0 10 0 0
0 10 0 10 10 0 0

Edges in MST:
0 -> 1 = 10
1 -> 2 = 10
4 -> 3 = 10
3 -> 5 = 10
2 -> 6 = 10
5 -> 4 = 10

Minimum Cost = 100
Process returned 0 (0x0)   execution time : 0.000 s
Press any key to continue.
```

(b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include <stdio.h>

struct Edge
{
    int u, v, weight;
};

int parent[10];

int find(int i)
{
    while(parent[i] != i)
    {
        i = parent[i];
    }
    return i;
}

void unionSet(int a, int b)
{
    parent[a] = b;
}

int main()
{
    int n, e;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
```

```

printf("Enter number of edges: ");
scanf("%d", &e);
struct Edge edge[e];
printf("Enter edges (u v weight):\n");
for(int i = 0; i < e; i++)
{
    scanf("%d %d %d",
        &edge[i].u,
        &edge[i].v,
        &edge[i].weight);
}
for(int i = 0; i < e - 1; i++)
{
    for(int j = i + 1; j < e; j++)
    {
        if(edge[i].weight > edge[j].weight)
        {
            struct Edge temp = edge[i];
            edge[i] = edge[j];
            edge[j] = temp;
        }
    }
}
for(int i = 0; i < n; i++)
{
    parent[i] = i;
}
int minCost = 0;
printf("\nEdges in MST:\n");

```

```

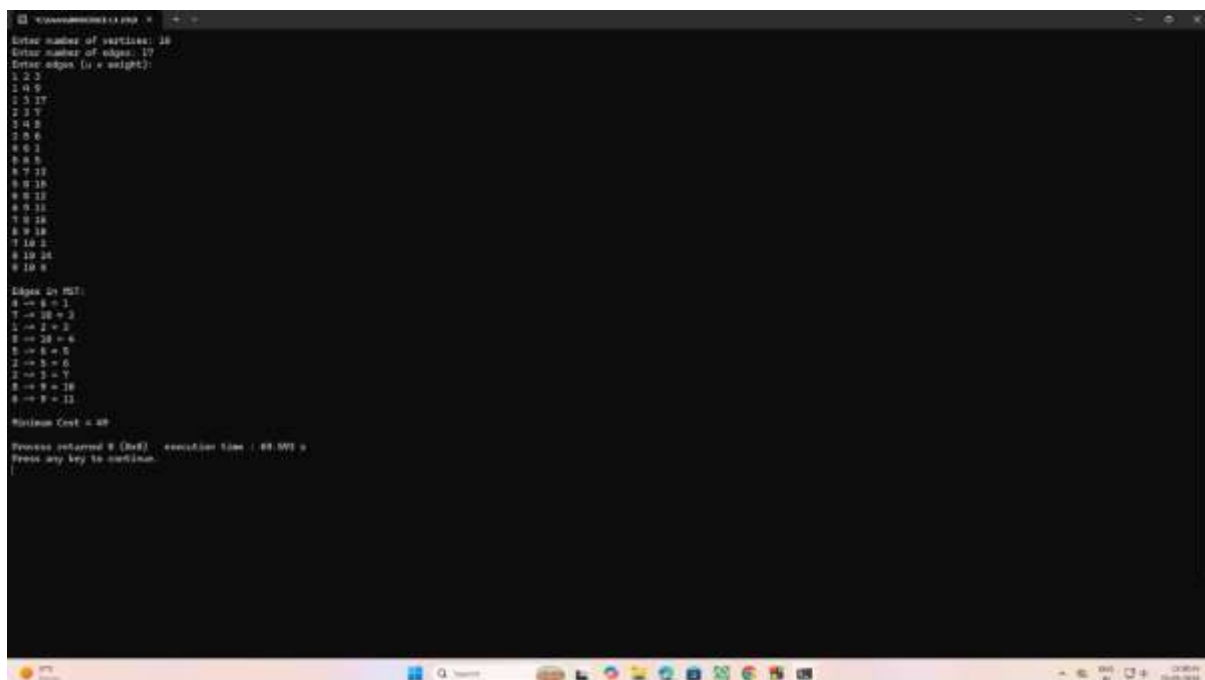
for(int i = 0; i < e; i++)
{
    int a = find(edge[i].u);
    int b = find(edge[i].v);
    if(a != b)
    {
        printf("%d -> %d = %d\n",
            edge[i].u,
            edge[i].v,
            edge[i].weight);
        minCost += edge[i].weight;
        unionSet(a, b);
    }
}

printf("\nMinimum Cost = %d\n", minCost);

return 0;
}

```

Output:



The screenshot shows a terminal window with the following output:

```

Enter number of vertices: 10
Enter number of edges: 17
Enter edges (u v weight):
1 2 3
1 4 5
1 3 17
2 3 7
3 4 8
3 5 6
4 6 1
5 6 5
6 7 11
6 8 10
6 9 11
6 9 11
7 8 16
8 9 10
7 10 1
8 10 16
9 10 8

Edges in MST:
4 -> 1 = 3
7 -> 10 = 1
1 -> 2 = 5
5 -> 10 = 6
5 -> 6 = 5
2 -> 3 = 6
2 -> 4 = 7
5 -> 9 = 10
6 -> 9 = 11

Minimum Cost = 48

Process returned 0 (0x0)   execution time : 0.000 s
Press any key to continue.

```

Lab Program 9:

Implement Fractional Knapsack using Greedy technique.

```
#include<stdio.h>

float w[50], p[50], ratio[50], x[50];

void knapsack(int m, int n)
{
    int rem_cap;
    float totalProfit = 0.0;
    for(int i = 0; i < n; i++)
    {
        x[i] = 0.0;
    }
    rem_cap = m;
    for(int i = 0; i < n; i++)
    {
        if(w[i] <= rem_cap)
        {
            x[i] = 1.0;
            rem_cap = rem_cap - w[i];
            totalProfit = totalProfit + p[i];
        }
        else
        {
            x[i] = (float)rem_cap / w[i];
            totalProfit =
            totalProfit + (x[i] * p[i]);
            break;
        }
    }
}
```

```

    }

    printf("\nSelected fractions:\n");
    for(int i = 0; i < n; i++)
    {
        printf("%.2f ", x[i]);
    }

    printf("\n");
    printf("Maximum Profit = %.2f\n", totalProfit);
}

int main()
{
    int n, m, i, j;
    printf("Enter number of items: ");
    scanf("%d", &n);
    printf("Enter weight and profit of each item:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%f %f", &w[i], &p[i]);
        ratio[i] = p[i] / w[i];
    }
    printf("Enter Maximum Capacity: ");
    scanf("%d", &m);
    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(ratio[i] < ratio[j])
            {

```

```

        float temp;

        temp = ratio[i];
        ratio[i] = ratio[j];
        ratio[j] = temp;

        temp = w[i];
        w[i] = w[j];
        w[j] = temp;

        temp = p[i];
        p[i] = p[j];
        p[j] = temp;
    }
}
}

knapsack(m, n);

return 0;
}

```

Output:

```

C:\Users\user> g++ knapsack.cpp
Enter number of items: 3
Enter weight and profit of each item:
10 20
15 30
20 15
Enter Knapsack Capacity: 20
Selected fractions:
1.00 0.50 0.00
Knapsack Profit = 32.50
Process returned 0 (0x0)   execution time : 01.166 s
Press any key to continue.

```

Lab Program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include<stdio.h>

#define MAX 10

#define INF 9999


int main()
{
    int n, i, j, start;
    int cost[MAX][MAX];
    int dist[MAX];
    int visited[MAX];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);

            if(cost[i][j] == 0 && i != j)
            {
                cost[i][j] = INF;
            }
        }
    }
    printf("Enter starting vertex (0 to %d): ", n - 1);
    scanf("%d", &start);
```



```

for(i = 0; i < n; i++)
{
    dist[i] = cost[start][i];

    visited[i] = 0;
}
dist[start] = 0;
visited[start] = 1;
for(i = 1; i < n; i++)
{
    int min = INF;
    int u = -1;
    for(j = 0; j < n; j++)
    {
        if(!visited[j] && dist[j] < min)
        {
            min = dist[j];
            u = j;
        }
    }
    if(u == -1)
        break;
    visited[u] = 1;
    for(j = 0; j < n; j++)
    {
        if(!visited[j] &&
            dist[u] + cost[u][j] < dist[j])
        {
            dist[j] =
                dist[u] + cost[u][j];

```

```

    }
}
}
printf("\nShortest distances from vertex %d:\n", start);
for(i = 0; i < n; i++)
{
    printf("To %d = %d\n", i, dist[i]);
}
return 0;
}

```

Output:

The screenshot shows a Windows command prompt window titled "C:\Users\user> g++ 01.cpp". The program prompts the user to enter the number of vertices (0), the cost matrix, and the starting vertex (0 to 5). The cost matrix is entered as a 6x6 grid of values. The program then outputs the shortest distances from vertex 0 to all other vertices.

```

C:\Users\user> g++ 01.cpp
Enter number of vertices: 6
Enter cost matrix:
0 0 11 9 8
22 0 15 0 6 9
1 0 0 0 0 0
0 0 0 0 0 0
0 0 7 1 0 0
20 2 0 0 1 0
Enter starting vertex (0 to 5): 0

Shortest distances from vertex 0:
To 0 = 0
To 1 = 2
To 2 = 9
To 3 = 4
To 4 = 1
To 5 = 8

Process returned 0 (0x0)   execution time : 34.000 s
Press any key to continue

```

Lab Program 11:

Implement “N-Queens Problem” using Backtracking.

```
#include<stdio.h>
```

```
#include<math.h>
```

```
int board[20], n;
```

```
int isSafe(int row, int col)
```

```
{  
    for(int i=1; i<=row; i++)  
    {  
        if(board[i]==col || (abs(board[i]-col) == abs(i-row)))  
            return 0;  
    }  
    return 1;  
}
```

```
void printSolution()
```

```
{  
    printf("\nSolution:\n");  
    for(int i=1; i<=n; i++)  
    {  
        for(int j=1; j<=n; j++)  
        {  
            if(board[i] == j)  
                printf("Q");  
            else
```

```

        printf(".");
    }
    printf("\n");
}
}

void solveNQueens(int row)
{
    for(int col=1; col<=n; col++)
    {
        if(isSafe(row, col))
        {
            board[row]=col;
            if(row==n)
                printSolution();
            else
                solveNQueens(row+1);
        }
    }
}

```

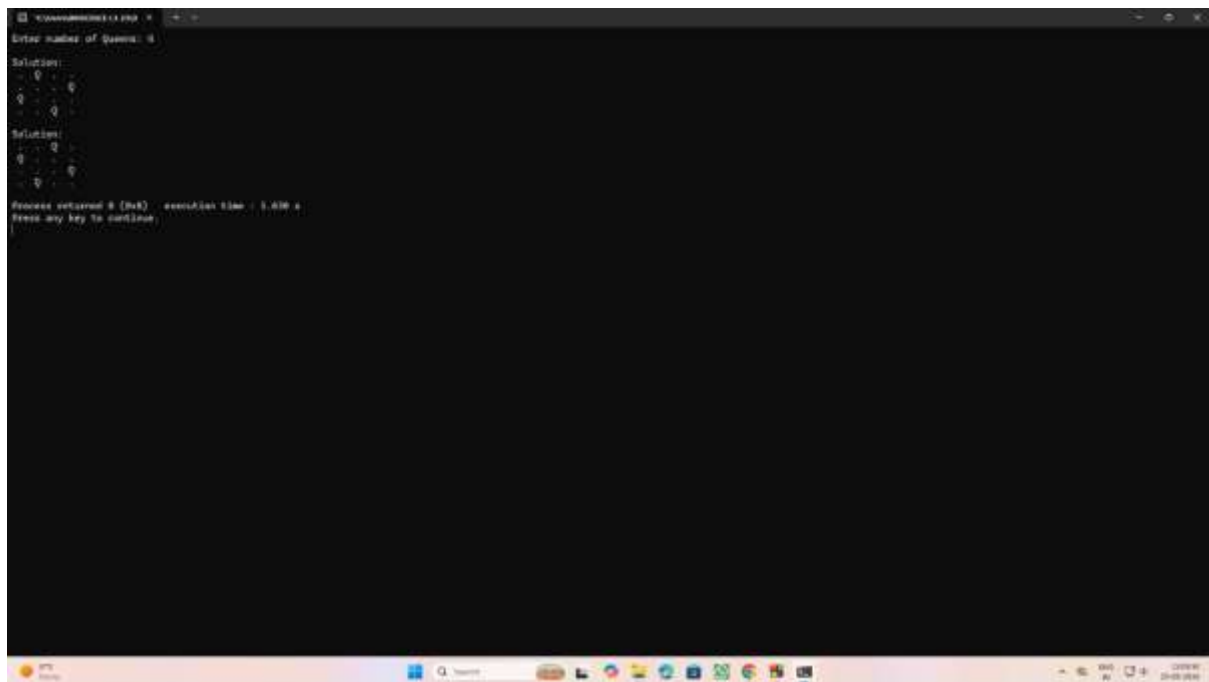
```

int main()
{
    printf("Enter number of Queens:");
    scanf("%d", &n);
    solveNQueens(1);
    return 0;
}

```

}

Output:



```
Enter number of Queens: 3
Solution:
  Q  -  - 
-  Q  -  - 
-  -  Q  - 
Solution:
  Q  -  - 
-  -  Q  - 
-  -  -  Q 
Process returned 0 (0x0)   execution time : 0.000 s
Press any key to continue
```